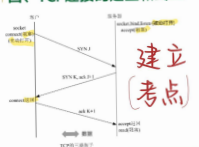


1. TCP的建立/终止 (考一个) (论述题)

四、TCP连接的建立和终止



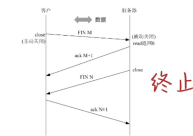
建立 考点

1. 服务器必须准备好接受外来的连接。这通过调用 `socket`、`bind` 和 `listen` 函数来完成，称为被动打开 (passive open)。
3. 服务器必须确认客户的 SYN，同时自己也得发送一个 SYN 分节。

2. 客户通过调用connect进行主动打开（active open）。这引起客户TCP发送一个SYN分节（表示请求）。

- | | | | | | | |
|---|---|---|---|---|---|--|
| U | A | P | B | L | F | |
| B | C | A | L | P | I | |
| G | H | I | J | N | N | |

- #### 4. 客户必须确认服务意图



终止

5. 某个应用程序员调用
用close，即这一端执行
主动关闭(active close)
}。
3. 一段时间后，接收到
文件描述符的应用进程
将调用close关闭它的套
接口。
4. 接收到这个FIN的原
送方TCP（即执行主动关
闭的那一端）对它进行
确认。
- 

2. 接收到 FIN 的另一端
并发送关闭 (cassive)



2. IP4 结构 (每個成員的含義) (簡答題)

1. IPv4套接口地址结构

```
struct in_addr { 内部IP地址结构体  
    in_addr_t s_addr; // 32位IPv4地址，网络字节序！  
};  
  
struct sockadd_in { IPv4套接字结构体  
    uint_t  
    sa_family_t    sin_len; // 结构体变量长度！  
    in_port_t      sin_family; // 协议！16位  
    struct in_port_t sin_port; // 16位端口号！  
    struct in_addr  sin_addr; IPv4地址  
    char            sin_zero[8]; // 保留！  
};
```

这个结构体类型的变量占用16字节内存。

当作为参数传递给任一个套接口函数时，套接口地址结构总是通过指针来传递，但通过指针来取得此参数的套接口函数必须处理来自所支持的任何协议族的套接口地址结构。

必考

3 程序阅读 $\rightarrow$ 说功能

二、简单的时间/日期客户程序

[illegible]

考阅读程序写功能

4. 套接口函数

socket(),

connect(),

bind(),

listen(),

accept();

5. 值—结果参数 (简答题)

当函数被调用时，结构大小是一个值（此值告诉内核该结构的大小，使内核在写此结构时不至于越界），当函数返回时，结构大小又是一个结果（它告诉进程，内核在此结构中确切存储了多少信息），这种参数类型叫值—结果参数。

6. UDP的 connect 函数 (简答题)

1. UDP connect 无三次握手：仅内核本地记录默认对端IP+端口，不产生任何网络报文交互，对端完全无感知，属于逻辑连接（伪连接）。

2. 本质等价于 setpeername：给UDP套接字绑定固定远端地址，建立「套接字-远端IP:端口」映射关系。

二、函数原型（考试直接写）

```
c
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

• 参数：

- sockfd：UDP套接字描述符（SOCK_DGRAM 类型）
- addr：目标服务器地址结构体（struct sockaddr_in）
- addrlen：地址结构体长度

• 返回值：成功返回0，失败返回-1并设置errno

一、bind error: Permission denied 原因与解决

原因：
Linux 系统中，端口号 1-1023 为特权端口，只有 root 权限的进程才能绑定。普通用户运行服务器时，若使用了 1023 以内的端口，就会触发权限拒绝错误。

解决方法：

1. **提升权限运行：**使用 `sudo ./daytimetcperv` 以 root 身份启动程序，即可绑定特权端口。
2. **更换为非特权端口：**将服务器和客户端代码中的端口号修改为 >1023 的整数（如 2015），普通用户无需额外权限即可绑定。

二、对 close 和 shutdown 函数的理解

• close 函数

- 作用：**完全关闭套接字**，释放文件描述符，同时关闭 TCP 连接的读写两端。
- 特点：会触发 TCP 四次挥手过程；若多个进程共享同一套接字，仅会减少引用计数，计数为 0 时才真正关闭。
- 局限：无法精细控制连接关闭方向，可能导致数据传输不完整。

• shutdown 函数

- 作用：**部分关闭套接字**，可单独关闭读端、写端或读写两端，不释放文件描述符。
- 特点：
 - SHUT_RD：关闭读端，无法再接收数据。
 - SHUT_WR：关闭写端，无法再发送数据（会触发 FIN 包）。
 - SHUT_RDWR：同时关闭读写两端，效果类似 close，但不释放文件描述符。
- 优势：支持优雅关闭（如先发送完数据再关闭写端，等待对方接收后再完全关闭）。

close与shutdown：

1) close将描述字的访问计数减1，仅在此计数为0时才关闭套接口。用shutdown我们可以激发TCP的正常连接终止序列，而不管访问计数。

2) close终止了数据传送的两个方向：读和写，shutdown可以只关闭读或写，也可以两个都关闭。

三、对 select 函数的理解

• 函数功能：

实现 I/O 多路复用，同时监控多个套接字（或文件描述符）的读、写、异常状态，当有描述符就绪时返回，避免阻塞在单个 I/O 操作上。

• 参数含义：

1. nfds：需要监控的最大文件描述符编号 +1。
2. readfds：可读事件的描述符集合。
3. writefds：可写事件的描述符集合。
4. exceptfds：异常事件的描述符集合。
5. timeout：超时时间，设为 NULL 则无限等待，设为 0 则立即返回。

• 返回值含义：

- >0：就绪的描述符数量。
- 0：超时，无描述符就绪。
- -1：调用出错（如被信号中断）。

四、对 TCP 并发服务器的理解

• 子进程创建：

父进程调用 `fork()` 创建子进程，子进程继承父进程的监听套接字和已连接套接字。父进程继续监听新连接，子进程负责处理已建立的客户端连接。

• 父子进程分工：

- **父进程：**专注于 `accept()` 接收新客户端连接，将连接套接字传递给子进程，自身不参与数据交互。
- **子进程：**关闭继承的监听套接字，与客户端进行数据收发，处理完业务后关闭连接并退出。

• 终止子进程的处理：

子进程退出后会变为僵尸进程，父进程需通过 `wait()` 或 `waitpid()` 回收其资源，避免资源泄漏。通常结合 `SIGCHLD` 信号处理函数，在子进程终止时自动回收。

五、对 socket、bind、listen、accept 和 connect 函数的理解

- **socket：**创建套接字文件描述符，指定协议族（如 IPv4）、类型（如 TCP 流套接字）和协议。
- **bind：**将套接字绑定到指定的 IP 地址和端口号，使客户端能定位到服务器。
- **listen：**将套接字设为被动监听状态，指定等待连接队列的长度，开始接收客户端连接请求。
- **accept：**从监听队列中取出一个已完成的连接，返回新的已连接套接字，用于与客户端通信。
- **connect：**客户端主动向服务器发起连接请求，建立 TCP 连接。

实验一

```
#include "unp.h"
#include <time.h>

int
main(int argc, char **argv)
{
    int listenfd, connfd;
    struct sockaddr_in servaddr;
    char buff[MAXLINE];
    time_t ticks;

    listenfd = Socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(13); /* daytime server */

    Bind(listenfd, (SA *) &servaddr, sizeof(servaddr));

    Listen(listenfd, LISTENQ);

    for ( ; ; ) {
        connfd = Accept(listenfd, (SA *) NULL, NULL);

        ticks = time(NULL);
        snprintf(buff, sizeof(buff), "%.24s\r\n", ctime(&ticks));
        Write(connfd, buff, strlen(buff));

        Close(connfd);
    }
}
```

图 1.9 TCP 时间/日期服务器程序

```
#include "unp.h"

int
main(int argc, char **argv)
{
    int sockfd, n;
    char recvline[MAXLINE + 1];
    struct sockaddr_in servaddr;

    if (argc != 2)
        err_quit("usage: a.out <IPaddress>");

    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
        err_sys("socket error");

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(13); /* daytime server */
    if (inet_pton(AF_INET, argv[1], &servaddr.sin_addr) <= 0)
        err_quit("inet_pton error for %s", argv[1]);

    if (connect(sockfd, (SA *) &servaddr, sizeof(servaddr)) < 0)
        err_sys("connect error");

    while ((n = read(sockfd, recvline, MAXLINE)) > 0) {
        recvline[n] = 0; /* null terminate */
        if (fputs(recvline, stdout) == EOF)
            err_sys("fputs error");
    }
    if (n < 0)
        err_sys("read error");

    exit(0);
}
```

图 1.5 TCP 时间/日期客户端程序

实验 =

服务器

加法

```
1 #include "unp.h"
2 #define SERV_PORT 9877
3 #define LISTENQ 1024
4 void str_echo(int sockfd);
5 void sig_chld(int signo);
6 int main()
7 {
8     int listenfd, connfd;
9     pid_t childpid;
10    socklen_t clien;
11    struct sockaddr_in cliaddr, servaddr;
12    listenfd = Socket(AF_INET, SOCK_STREAM, 0);
13    bzero(&servaddr, sizeof(servaddr));
14    servaddr.sin_family = AF_INET;
15    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
16    servaddr.sin_port = htons(SERV_PORT);
17    Bind(listenfd, (SA *)&servaddr, sizeof(servaddr));
18    Listen(listenfd, LISTENQ);
19    Signal(SIGCHLD, sig_chld);
20    for (;;) {
21        clien = sizeof(cliaddr);
22        if ((connfd = accept(listenfd, (SA *)&cliaddr, &clien)) < 0) {
23            if (errno == EINTR) continue;
24            else err_sys("accept error");
25        }
26        if ((childpid = Fork()) == 0) {
27            Close(listenfd);
28            str_echo(connfd);
29            exit(0);
30        }
31        Close(connfd);
32    }
33 }
34 void str_echo(int sockfd)
35 {
36     long arg1, arg2;
37     ssize_t n;
38     char line[MAXLINE];
39     for (;;) {
40         if ((n = Readline(sockfd, line, MAXLINE)) == 0) return;
41         if (sscanf(line, "%ld%ld", &arg1, &arg2) == 2)
42             sprintf(line, sizeof(line), "%ld\n", arg1 + arg2);
43         else
44             sprintf(line, sizeof(line), "input error\n");
45         n = strlen(line);
46         Writen(sockfd, line, n);
47     }
48 }
49 void sig_chld(int signo)
50 {
51     pid_t pid;
52     int stat;
53     while ((pid = waitpid(-1, &stat, WNOHANG)) > 0)
54         printf("child %d terminated\n", pid);
55 }
```

```
1 #include "unp.h"
2 #define SERV_PORT 9877
3 #define LISTENQ 1024
4 void str_echo(int sockfd);
5 void sig_chld(int signo);
6 int main()
7 {
8     int listenfd, connfd;
9     pid_t childpid;
10    socklen_t clien;
11    struct sockaddr_in cliaddr, servaddr;
12    listenfd = Socket(AF_INET, SOCK_STREAM, 0);
13    bzero(&servaddr, sizeof(servaddr));
14    servaddr.sin_family = AF_INET;
15    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
16    servaddr.sin_port = htons(SERV_PORT);
17    Bind(listenfd, (SA *)&servaddr, sizeof(servaddr));
18    Listen(listenfd, LISTENQ);
19    Signal(SIGCHLD, sig_chld);
20    for (;;) {
21        clien = sizeof(cliaddr);
22        if ((connfd = accept(listenfd, (SA *)&cliaddr, &clien)) < 0) {
23            if (errno == EINTR) continue;
24            else err_sys("accept error");
25        }
26        if ((childpid = Fork()) == 0) {
27            Close(listenfd);
28            str_echo(connfd);
29            exit(0);
30        }
31        Close(connfd);
32    }
33 }
34 void str_echo(int sockfd)
35 {
36     ssize_t n;
37     char line[MAXLINE];
38     for (;;) {
39         if ((n = Readline(sockfd, line, MAXLINE)) == 0) return;
40         Writen(sockfd, line, n);
41     }
42 }
43 void sig_chld(int signo)
44 {
45     pid_t pid;
46     int stat;
47     while ((pid = waitpid(-1, &stat, WNOHANG)) > 0)
48         printf("child %d terminated\n", pid);
49 }
```

客户端

```
1 #include "unp.h"
2 #define SERV_PORT 9877
3 void str_cli(FILE *fp, int sockfd);
4 int main(int argc, char **argv)
5 {
6     int sockfd;
7     struct sockaddr_in servaddr;
8     if (argc != 2) err_quit("usage: tcpcli <IPaddress>");
9     sockfd = Socket(AF_INET, SOCK_STREAM, 0);
10    bzero(&servaddr, sizeof(servaddr));
11    servaddr.sin_family = AF_INET;
12    servaddr.sin_port = htons(SERV_PORT);
13    Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
14    Connect(sockfd, (SA *)&servaddr, sizeof(servaddr));
15    str_cli(stdin, sockfd);
16    exit(0);
17 }
18 void str_cli(FILE *fp, int sockfd)
19 {
20     char sendline[MAXLINE], recvline[MAXLINE];
21     while (Fgets(sendline, MAXLINE, fp) != NULL) {
22         Writen(sockfd, sendline, strlen(sendline));
23         if (Readline(sockfd, recvline, MAXLINE) == 0)
24             err_quit("str_cli: server terminated prematurely");
25         Fputs(recvline, stdout);
26     }
27 }
```

实验三

服务器

客户端

```
1 #include "unp.h"
2 #define SERV_PORT 9877
3 #define LISTENQ 1024
4 int main()
5 {
6     int i, maxi, maxfd, listenfd, connfd, sockfd;
7     int nready, client[FD_SETSIZE];
8     fd_set rset, allset;
9     char line[MAXLINE];
10    socklen_t clien;
11    struct sockaddr_in cliaddr, servaddr;
12    // 1. 创建监听套接字
13    listenfd = Socket(AF_INET, SOCK_STREAM, 0);
14    bzero(&servaddr, sizeof(servaddr));
15    servaddr.sin_family = AF_INET;
16    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
17    servaddr.sin_port = htons(SERV_PORT);
18    Bind(listenfd, (SA *)&servaddr, sizeof(servaddr));
19    Listen(listenfd, LISTENQ);
20    // 2. 初始化 select 相关变量
21    maxfd = listenfd;
22    maxi = -1;
23    for (i = 0; i < FD_SETSIZE; i++)
24        client[i] = -1;
25    FD_ZERO(&allset);
26    FD_SET(listenfd, &allset);
27    // 3. 主循环
28    for (;;) {
29        rset = allset;
30        nready = Select(maxfd + 1, &rset, NULL, NULL, NULL);
31        // 有新连接
32        if (FD_ISSET(listenfd, &rset)) {
33            clien = sizeof(cliaddr);
34            connfd = Accept(listenfd, (SA *)&cliaddr, &clien);
35            // 找一个空闲位置存新连接
36            for (i = 0; i < FD_SETSIZE; i++) {
37                if (client[i] < 0) {
38                    client[i] = connfd;
39                    break;
40                }
41            }
42            if (i == FD_SETSIZE) err_quit("too many clients");
43            // 更新 select 集合和最大描述符
44            FD_SET(connfd, &allset);
45            if (connfd > maxfd) maxfd = connfd;
46            if (i > maxi) maxi = i;
47            if (--nready < 0) continue;
48        }
49        // 处理客户端数据
50        for (i = 0; i <= maxi; i++) {
51            if ((sockfd = client[i]) < 0) continue;
52            if (FD_ISSET(sockfd, &rset)) {
53                int n = Readline(sockfd, line, MAXLINE);
54                if (n == 0) {
55                    // 客户端断开
56                    Close(sockfd);
57                    FD_CLR(sockfd, &allset);
58                    client[i] = -1;
59                } else {
60                    // 回射
61                    Writen(sockfd, line, n);
62                }
63                if (--nready <= 0) break;
64            }
65        }
66    }
67 }
```

```
1 #include "unp.h"
2 #define SERV_PORT 9877
3 void str_cli(FILE *fp, int sockfd);
4 int main(int argc, char **argv)
5 {
6     int sockfd;
7     struct sockaddr_in servaddr;
8     if (argc != 2) err_quit("usage: tcpcli <IPaddress>");
9     sockfd = Socket(AF_INET, SOCK_STREAM, 0);
10    bzero(&servaddr, sizeof(servaddr));
11    servaddr.sin_family = AF_INET;
12    servaddr.sin_port = htons(SERV_PORT);
13    Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
14    Connect(sockfd, (SA *)&servaddr, sizeof(servaddr));
15    str_cli(stdin, sockfd);
16    exit(0);
17 }
18 void str_cli(FILE *fp, int sockfd)
19 {
20     int maxfdp1, stdineof = 0;
21     fd_set rset;
22     char sendline[MAXLINE], recvline[MAXLINE];
23     FD_ZERO(&rset);
24     for (;;) {
25         if (stdineof == 0) FD_SET(fileno(fp), &rset);
26         FD_SET(sockfd, &rset);
27         maxfdp1 = max(fileno(fp), sockfd) + 1;
28         Select(maxfdp1, &rset, NULL, NULL, NULL);
29         // 收到服务器数据
30         if (FD_ISSET(sockfd, &rset)) {
31             int n = Readline(sockfd, recvline, MAXLINE);
32             if (n == 0) {
33                 if (stdineof == 1) return;
34                 else err_quit("server terminated prematurely");
35             }
36             Fputs(recvline, stdout);
37         }
38         // 收到用户输入
39         if (FD_ISSET(fileno(fp), &rset)) {
40             if (Fgets(sendline, MAXLINE, fp) == NULL) {
41                 stdineof = 1;
42                 Shutdown(sockfd, SHUT_WR);
43                 FD_CLR(fileno(fp), &rset);
44                 continue;
45             }
46             Writen(sockfd, sendline, strlen(sendline));
47         }
48     }
49 }
```

函数功能

函数	核心功能	特点/局限
gethostname	主机名 → IPv4 地址	仅 IPv4, 线程不安全
gethostname2	主机名 → 指定地址族地址	支持 IPv6, 仍线程不安全
gethostbyname	IP 地址 → 主机名	反向解析, 线程不安全
uname	获取完整系统内核信息	包含主机名、系统版本等
gethostnamel	仅获取主机名	轻量, 功能单一
getservbyname	服务名 → 端口号	查询服务映射
getservbyport	端口号 → 服务名	反向查询服务映射

select(), 允许进程指示内核等待多个事件, 并统一发生, 有仅一个或多个事件发生或超时时间后无事件发生

shutdown(), ~

poll(), I/O 多复用机制, 同时支持多个文件描述符的 I/O 事件

epoll(), 高性能 I/O 多路复用机制, 相比 select/poll 效率更高

pread(), select() 的升级版

recvfrom(), 面向无连接套接字的核心接收函数

recvmsg(), 用于接收套接字接收数据并获取发这方的地址信息

bind(), 将本地 IP + 端口绑定到套接字, 用于服务器固定监听地址。

connect(), 客户端主动发起与服务器的连接, 建立通信链路。

sendto(), 发送 UDP 数据报, 需显式指定目标地址。

accept(), 服务器接受新 TCP 连接, 并获取远端地址。

recvfrom(), 接收 UDP 数据报, 并获取发送方源地址。

getsockname(), 获取套接字绑定的本地 IP + 端口。

getpeername(), 获取已连接套接字对应的远端 IP + 端口。

readn(), 保证读取指定长度的字节

writen(), 保证写入指定长度的字节

readline(), 按行读取数据, 以描述符缓冲区读取, 直到遇到换行符 \n, 达到 wbuffer - 1 位置, 输出 EOF。

isfdtype(), 测试一个描述符是否是某种数据类型。

socket(), 在内核中创建一个套接字对象, 并返回一个非整数的文件描述符 (类似返回 -1)。

listen(), 接收 TCP 服务器调用, 该函数 socket() 创建一个套接口时, 它被假设为一个主动套接口, 是一个将调用

accept() 发起连接的客户套接口, 该函数 listen() 将未连接的套接口转换成被动套接口, 指示内核应接受指向此套接口的连接请求。

(2) 该函数第二个参数规定了内核为此套接口排队等待的最大连接个数。

accept(), 由 TCP 服务器调用, 从已完成连接队列中返回下一个已完成连接, 若该函数被阻塞, 则进入睡眠。

fork(), ① 一个进程可以为自己创建一个拷贝, 这样一个拷贝处理一个操作时, 其他拷贝仍做其他任务。

② 一个进程想执行新操作, 由于创建新进程的唯一方法是调用 fork, 进程调用 fork 来生成一个拷贝, 然后其中一个拷贝调用 execl 来代替自己在执行新操作。

close(), 用关闭套接字描述符的系统调用, 并做进程与套接字之间的关联, 并根据描述符引用次数决定是否发送 TCP 连接终止。

wait(), 任意终止进程, 强制阻塞, 低, 只能简单等待

waitpid(), 可指定 PID 进程组/任意子进程, 高, 可精确控制等待行为

父进程回收子进程资源的系统调用

核心目的是避免子进程变成僵尸进程。